POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Executive Summary of the Thesis

From Historical F1 Strategy Streams to Pit Stop Decision Models

Laurea Magistrale in Computer Science and Engineering - Ingegneria Informatica

Author: Pietro Pizzoccheri

Advisor: Prof. Emanuele Della Valle

Academic year: 2025-26

---

1. Introduction

Formula 1 race strategy is a decision process in which information changes faster than the decision can be comfortably analysed. A pit stop can recover tyre performance, answer an undercut, or protect track position; it can also return the driver into traffic, waste a tyre advantage, or make the car vulnerable later in the race. The same action can therefore be correct or wrong depending on stint age, rival gaps, pit-loss estimate, track status, weather, race phase, and the expected behaviour of nearby opponents.

This thesis studies whether historical Formula 1 races can be transformed into a **reproducible, temporally valid decision support problem** for pit stop calls. Historical races are extracted with FastF1, enriched into replayable event records, streamed through Kafka, processed by Apache Flink, and converted into learning datasets for Batch and online models.

The work is organised around **two prediction contracts**:

- `pit_any_h2`: a timing contract, positive when a driver pits within the next two laps.
- `pit_success_h2`: a stricter recommendation contract, positive only when the future pit stop is also classified as successful.

The research questions follow from this distinction. First, can near term pit timing be learned from race state patterns while respecting temporal order? Second, can successful pit calls be evaluated under a stricter operational contract, where positive recommendations without supporting future evidence are penalised?

The contribution is mainly methodological and practical:

- a **complete pipeline** from race extraction to streaming replay, stateful processing, model training, online evaluation, and contract aware comparison

- two explicit pit decision contracts with horizon $H = 2$
- a comparison between a deterministic Flink Strategy Engine, Batch XGBoost models, and MOA online learners
- an evaluation protocol that separates row-level precision, event coverage, strict operational precision, and diagnostic matched pit quality

Keywords: Formula 1 strategy, event stream processing, Apache Flink, Kafka, pit stop prediction, online learning, imbalanced classification.

2. Background and State of the Art

The thesis is positioned at the intersection of stream processing, motorsport strategy, machine learning on tabular race data, online learning, and imbalanced evaluation. Stream processing provides the technical basis for replaying a historical race as a sequence of events rather than as a static table. In this setting, event time, watermarks, keyed state, and windowed computations are essential concepts: a decision system must react to **what is known at that instant**. Apache Flink is therefore a relevant foundation for the work [2], while Kafka provides the replay log used by the implementation.

Formula 1 strategy adds a domain constraint to this streaming view. Pit stop decisions are affected by tyre degradation, pit loss estimates, safety car and virtual safety car periods, traffic, gaps to rivals, track position, compound choice, and weather. A pit call is also strategic rather than purely predictive: the same lap may be attractive for one driver because of an undercut opportunity and unattractive for another because of a poor rejoin position. This makes the problem different from a simple future event forecast.

The data source used by the thesis is FastF1, which exposes historical timing, lap, telemetry, weather, stint, and session information [4]. These sources are valuable but not already aligned for live like decision support. They must be cleaned, joined, enriched, and transformed into records that preserve the order in which information would have been observed during a race.

On the modelling side, the thesis compares three families of systems. The first is a **deterministic Flink Strategy Engine**, designed as an interpretable rule based baseline. The second is a **Batch learner based on XGBoost**, suitable for nonlinear interactions in tabular features and for post hoc feature attribution [3]. The third is an **online learner evaluated through MOA**, using a prequential test-then-train protocol and an OzaBoostADWIN learner [1]. Batch learning is allowed to learn from previous races before scoring held-out races; online learning updates sequentially after each evaluated instance.

A central difficulty is **class imbalance**. Pit opportunities are rare compared with ordinary driver-lap records, and successful pit opportunities are rarer still. Accuracy is therefore not a meaningful primary measure: a system can be highly accurate by almost never recommending a pit stop. The evaluation instead relies on precision, recall or event coverage, $F_{0.5}$, average precision, Cohen's Kappa, and G-Mean, with particular attention to the cost of false positive pit calls.

3. Problem Statement and Data

The core problem is to transform historical race data into decision instances that are **comparable across systems** and **do not use future information**. The raw data contain lap times, sector times, posi-

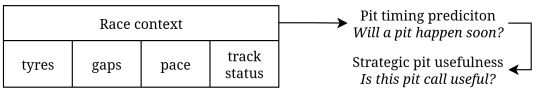


Figure 1: The two target contracts: near-term pit occurrence and near-term successful pit recommendation.

tions, stint data, pit stop evidence, track status, weather, and timing gaps. These observations are not sufficient on their own: a strategy system needs race context, including driver state, rival state, session state, tyre age, recent pace loss, possible drop zone, race progress, and weather conditions.

The decision unit is a **driver-lap state**. At that state, the system may emit a positive pit recommendation. Exact pit lap prediction is avoided because exact timing is too brittle: a call one lap early may still be operationally meaningful. Both contracts therefore use a **horizon** $H = 2$.

The two contracts create different evaluation problems:

- in `pit_any_h2`, a positive call is correct when it can be matched to an eligible future pit event;
- in `pit_success_h2`, a positive call is correct only when the matched future pit stop is classified as successful;
- calls with no matched future pit, or calls matched to failed pit stops, are false positives under the success contract.

The evaluation protocol is built around a **shared truth universe**. Systems are compared only over race-driver contexts for which the necessary truth information and model outputs can be aligned. A clean actionable lens removes cases that make the operational contract ambiguous, such as unresolved or noisy pit outcomes.

This separation is important in a rare positive setting since a model may achieve good

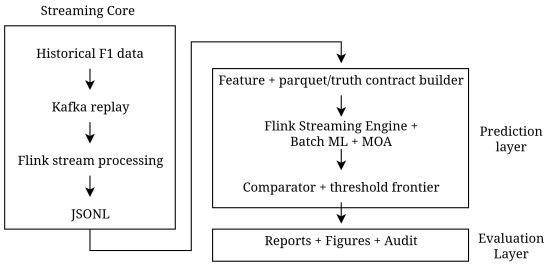


Figure 2: Pipeline architecture from historical race extraction to streaming, learning, and comparison.

ranking scores while recommending very few pit stops, or it may cover many events by producing too many false alarms. The protocol therefore distinguishes row-level precision, event coverage, **strict operational precision**, and matched-pit success rate. The last quantity is kept only as a diagnostic, because it ignores recommendations that never match a real pit stop.

4. Solution and Design

The implemented solution is organised into **three layers** (Figure 2). The Streaming Core Layer extracts historical race data, creates replayable event records, streams them through Kafka, and processes them with Flink. The Evaluation Modelling Layer converts persisted stream artifacts into Parquet datasets, builds the target contracts, trains and evaluates the Batch and MOA learners, and aligns the systems for comparison. The Delivery Layer produces reports, figures, audit tables, and validation outputs.

The ingestion stage starts from a race preparation script. FastF1 sessions are loaded and converted into a race snapshot containing lap events, telemetry related observations, track status changes, weather, stint information, pit stop evidence, and race metadata. Lap events are enriched with team and stint

data, total laps, pit loss estimates, and available gap information. Events are then sorted by absolute timestamp and replayed through Kafka topics, so the historical race becomes an ordered log.

The Flink job consumes the replay topics and builds the **live race context** with **event-time processing**. Track status is broadcast to the operators that need it, while driver streams are keyed by race and driver. This allows stateful operators to maintain stint-specific and driver specific information. Windowed computations group cars by race and lap to identify rivals ahead and behind, compute relevant gaps, and emit machine learning feature rows as side outputs.

The stateful strategy logic produces several interpretable artifacts:

- a tyre drop detector, which keeps the best lap in the current stint and counts consecutive slow laps
- a drop-zone evaluator, which estimates where the driver would rejoin after a pit stop
- a pit stop evaluator, which labels real pit outcomes from post-pit evidence
- a pit suggestion stream, which scores whether the current context supports MONITOR, GOOD_PIT, or PIT_NOW

The **Flink Strategy Engine** is the deterministic baseline. Its score combines pace loss, track status, traffic, strategic pressure, actionability, timing, end-of-race effects, competitive advantage, and uncertainty penalties. The final profile used in the evaluation is a guarded variant: rival pressure can increase urgency, but only when timing, gap, and traffic conditions support the call.

The **Batch path** uses the same artifacts to build supervised learning matrices for XGBoost. The matrix builder removes leakage-prone columns such as target labels, matched pit evidence, truth-lens flags, eligibility indicators, and metadata. Two feature profiles are compared: No-Year, which removes `source_year` after it was identified as a shortcut risk, and Percent, which emphasises race-relative quantities such as race progress and lap-time ratios.

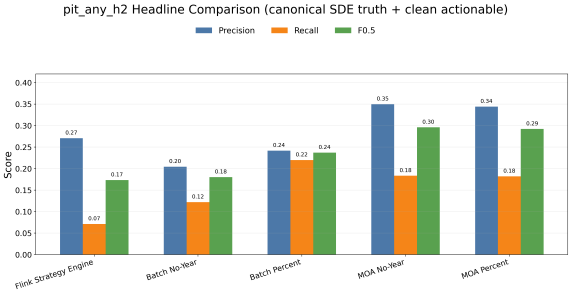
The **online path** exports the shared matrix logic to ARFF and evaluates MOA learners prequentially. Each instance is first scored, then used for training. The Java vote logger records labels, hard predictions, class votes, and a positive score derived from the vote distribution. These outputs are reconstructed with metadata so that MOA can be compared with Batch and Flink under the same threshold and event-matching logic.

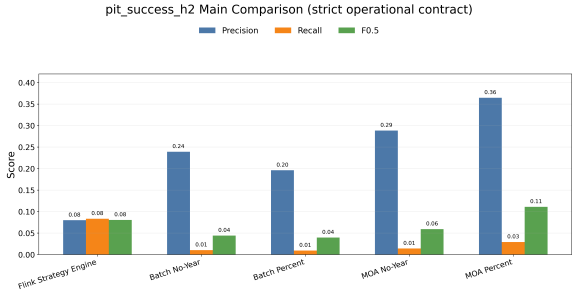
A substantial part of the design is devoted to **fairness corrections**. The comparator aligns systems on the same race-driver universe, applies the same horizon, distinguishes raw and clean truth lenses, and evaluates both row-level and event-level behaviour. This prevents an implementation detail, such as missing outputs for one driver or a different denominator, from becoming a misleading performance gain.

5. Experimental Evaluation

The frozen evaluation uses **Formula 1 races from 2022 to 2025**, the `canonical_sde_truth` universe, the `clean_actionable` lens, horizon $H = 2$, and five systems: the Flink Strategy Engine, Batch No-Year, Batch Percent, MOA No-Year, and MOA Percent. Batch systems are evaluated with race-sequential out-of-fold predictions. MOA systems are evaluated prequentially. Flink is evaluated through the persisted pit suggestion stream.

For `pit_any_h2`, the learner matrix contains 93,623 driver-lap rows, of which 5,855 are positive, about 6.3%. The shared com-





	`pit_any_h2`			`pit_success_h2`		
System	Precision	Coverage	$F_{0.5}$	Precision	Coverage	$F_{0.5}$
Flink Strategy Engine	0.271	0.071	0.173	0.080	0.083	0.081
Batch No-Year	0.204	0.122	0.180	0.239	0.010	0.044
Batch Percent	0.242	0.220	0.237	0.196	0.009	0.040
MOA No-Year	0.350	0.183	0.296	0.288	0.014	0.059
MOA Percent	0.344	0.182	0.292	0.365	0.029	0.111

Figure 3: Headline comparison for `pit_any_h2` (left) and strict `pit_success_h2` (right) under the clean actionable protocol. The table reports the selected operating points; coverage means event recall for `pit_any_h2` and successful-event coverage for `pit_success_h2`.

parator event universe contains 3,048 unique eligible pit events. The selected operating points in Figure 3 show that **near-term pit timing is learnable**: the deterministic Flink baseline is useful but conservative, Batch Percent reaches the largest event coverage, and MOA gives the strongest precision-oriented balance. The precision-recall analysis supports the same conclusion. Average precision for `pit_any_h2` is 0.131 for Batch No-Year, 0.190 for Batch Percent, 0.161 for MOA No-Year, and 0.195 for MOA Percent. The **Percent feature profile** therefore improves ranking quality for the timing task, especially by replacing absolute or identity like variables with race-relative signals.

For `pit_success_h2`, the results are more limited and more informative. The **strict contract** counts unmatched positive calls and failed matched pits as false positives. Under this rule, **MOA Percent** is the strongest selected system, but its successful-event coverage remains low. A relaxed MOA Percent point raises success coverage to 0.169 with precision 0.205, confirming that successful pit prediction is sparse and threshold sensitive: more coverage is possible, but only by accepting more false positives.

The Flink diagnostics further clarify why strict precision is necessary. For PIT_NOW alerts, the matched-pit success rate is 0.583, but strict precision is only 0.080. When GOOD_PIT alerts are added, the matched-pit success rate increases to 0.628 while strict precision decreases to 0.060. **Matched-pit success rate** describes only recommendations that eventually match a real pit stop; it ignores recommendations that never match any pit evidence.

Runtime measurements show the expected engineering trade off. Batch No-Year re-

quires about 1 minute and 44 seconds for `pit_any_h2` and 1 minute and 41 seconds for `pit_success_h2`; Batch Percent is slightly faster. MOA runs in a single pass and requires about 18–24 seconds for the target/profile combinations.

Race level Kappa and G-Mean provide a view of stability across events. For `pit_any_h2`, Batch Percent and MOA produce more consistent positive behaviour than the deterministic baseline, although results still oscillate by race. For `pit_success_h2`, the support is smaller and race level metrics are noisier.

Finally, explainability is treated differently for each system. Flink is transparent by construction because its score components and gates are explicit. Batch XGBoost is analysed with SHAP, which confirms the importance of race context variables and supports the removal of shortcut features. MOA scores are vote derived ranking scores rather than calibrated probabilities, and faithful online explanations are not retained in the final protocol.

6. Conclusions

The thesis shows that pit stop decision support can be studied from historical Formula 1 data only if the problem is made **temporal, contractual, and comparable**. Historical records must be replayed or reconstructed in event order; the target must state exactly what a correct positive recommendation means; and systems must be evaluated on the same truth universe. Without these constraints, the comparison between a rule engine, a Batch learner, and an online learner would mix modelling quality with leakage, denominator differences, or post-race information.

The first research question receives a positive answer. Near-term pit timing, represented by `pit_any_h2`, is a **learnable task**. The learning systems improve over the deterministic baseline, with Batch Percent providing the largest event coverage and MOA offering the best precision-oriented operating points. The result is not that pit stops can be predicted perfectly, but that race-state patterns contain enough information to support better-than-rule-baseline timing recommendations under a live-like chronology.

The second research question receives a more cautious answer. Successful pit recommendation, represented by `pit_success_h2`, is **much harder**. MOA Percent obtains the cleanest selected strict operating point, but coverage remains low. Relaxed thresholds can recover more events, at the cost of more false positives. This confirms that successful-pit prediction is not simply pit-timing prediction with another label: it depends on later race development, rival reactions, traffic, tyre performance, and the imperfect heuristic used to classify pit outcomes.

The main limitations follow from the experimental nature of the work. The replay is controlled and reproducible, but it is not a production live feed with all latency, correction, missing-data, and failure modes. The PitStopEvaluator is a useful approximation of outcome quality, not a full strategic utility model. Success labels are sparse, so race-level diagnostics can be unstable. Thresholds strongly influence the apparent behaviour of each system. Finally, only a limited set of learners is evaluated: XGBoost for Batch learning and OzaBoostADWIN for MOA online learning.

Future work should therefore move in four directions. First, the pipeline should be connected to a live ingestion setting, so that delay and missing-data behaviour can be evaluated directly. Second, the truth model should be refined through richer outcome definitions, probabilistic labels, or expert validation. Third, the learning problem

could move from row-level classification toward event-level ranking, time-to-pit modelling, or sequence-aware policies under a fixed false-positive budget. Fourth, online learning should be expanded with stronger calibration and faithful online explainability. Overall, the contribution of the thesis is an **executable and auditable framework** for asking a precise question: when a system says "pit now", what contract is it satisfying, what information was available, and how should that call be counted? The answer is strongest for pit timing and still preliminary for successful pit recommendation, but the pipeline makes both problems measurable in a reproducible way.

Bibliography

[1] Albert Bifet, Geoff Holmes, Richard Kirkby, and Bernhard Pfahringer. Moa: Massive online analysis. *Journal of Machine Learning Research*, 11:1601–1604, 2010.

[2] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 38(4):28–38, 2015.

[3] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[4] T. Oehrly. Fastf1: A python package for accessing and analyzing formula 1 results, schedules, timing data, and telemetry, 2025. Python package.